

Simulating LLaMA 3b on an Event-based TPU Simulator

Presented by Tejus Singh, Amaan Mohammad, Nikhita Neelakanta

Motivations

- Before an ML accelerator is built, we want to simulate running the target workload on the proposed chip, to see the performance and timing.
 - Test architecture configurations, memory-bound, compute-bound, bottlenecks, etc. See what direction to pursue.
- As such, hardware simulators are built to stress test, benchmark, and debug, to simulate overall architecture before even writing RTL and designing
- There are two types of simulators: event-based and cycle-based
 - event -based simulators process changes in the state, via events. Events carry an intrinsic latency, and dependency on other events. As such, you can keep track of timing within the cycle, but fast track to next event, making simulation timing fast.
 - Cycle based simulators treat the clock cycle as the main timekeeper, and trigger at each clock cycle edge. Slower and more accurate

Event-driven Simulator

Amaan Slides to cover

- Motivation
- Simulator how it works
 - Flexibility - layer, schedule, model, unit, registering instructions
- Reservation Stations for Unit Pools and Flexibility
 - Transformation of scheduler from 1 attention to overlapped layers
- Hw units architecture (diagram)
 - Systolic
 - Vector
 - Access
 - Dma
 - buffer
- HW Config + Diagram
- How to add new units - ex sparse unit

The Simulator

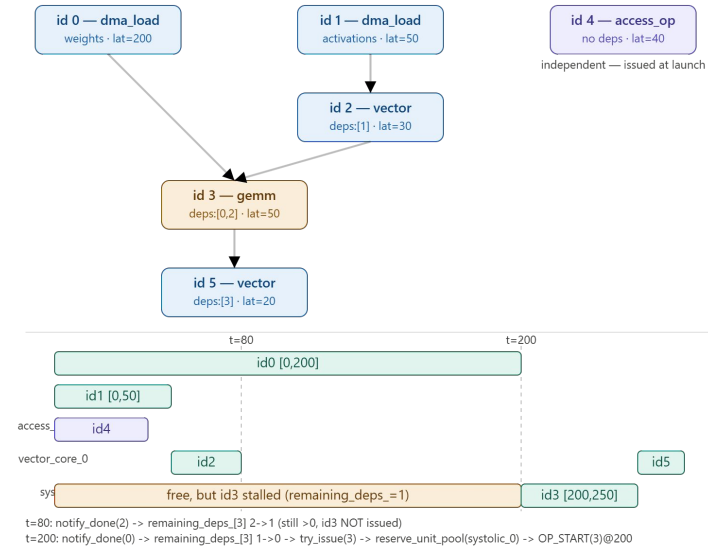
- C++-based event-driven simulator using min-heap
 - Timing-only not functional
 - Not a study on how to multiply matrices
 - How long
 - Jump cycles, next event at 50,000 cycles go there
 - Models events
 - Yaml arch config
 - Instruction static schedule (decided beforehand)
 - Hand-written
 - CPP generated
 - Main blocks -> Hardware units, Operations that use them, dependencies, and computed latencies per unit
- Simulator Engine Loop
 - Ex. LLaMa3 workload -> llama_schedule.cpp creates DAG data structure (very large)
 - Schedule converted into 3 arrays

```
event_engine.h x event_engine.cpp arch_config.h ! default.yaml Extension: Mar > <
src > core > C event_engine.h > {} sim > EventEngine
12 namespace sim {
22 class EventEngine {
80
90 private:
91     using MinHeap = std::priority_queue<Event, std::vector<Event>, std::greater<Event>>;
92 }
```

```
126 EventId EventEngine::schedule(Event e) {
127     if (e.cycle < now_) {
128         throw std::runtime_error(
129             "EventEngine::schedule: event at cycle " + std::to_string(e.cycle)
130             " is in the past (now=" + std::to_string(now_) + ")");
131     if (e.seq == 0) e.seq = next_seq++;
132     EventId id = e.seq;
133     queue_.push(std::move(e));
134     return id;
135 }
```

```
142 Cycle EventEngine::run(Cycle stop_at) {
143     while (!queue_.empty()) {
144         if (queue_.top().cycle > stop_at) break;
145
146         Event e = queue_.top();
147         queue_.pop();
148         now_ = e.cycle;
149
150         if (trace_) trace_(e);
151
152         Unit* u = get_unit(e.target);
153         if (u) u->handle(e, *this);
154     }
155     return now_;
156 }
157
158 } // namespace sim
159
```

- Schedule (full, upfront DAG) -> mapped by indices, my dependencies, who to notify, instruction I map to
 - -> Scheduler ctor: compact DAG into remaining_deps_/successors_/by_id_
 - -> launch(): try_issue() every 0-dep instruction
 - -> try_issue(): dispatch to op handler
 - -> handler: reserve_unit_pool() picks free unit @ earliest cycle, COMPUTES LATENCY, books availability
 - -> handler: schedule OP_START event at reserved cycle
 - -> engine.run(): pop min(cycle,seq), dispatch to unit
 - -> unit OP_START: schedule OP_DONE at start+latency
 - -> unit OP_DONE: notify_done() -> decrement successors' remaining_deps_ -> try_issue() newly-ready
 - -> repeat until heap empty / all_done()



```
Event e;  
e.type = EventType::OP_START;  
e.target = res.id;  
e.cycle = res.start;  
e.instr = ctx.inst.id;  
e.label = ctx.inst.label;  
e.payload = static_cast<int64_t>(0);  
ctx.engine.schedule(std::move(e));
```

```

PRESENTATION.md U ! gqa_fa2_overlapped.yaml U ! gqa_fa2_single.yaml U
schedules > ! gqa_fa2_single.yaml
# tile j+1 (stage 0) depends on tile j (accumulate) -> no inner-loop
# * head h+1 (load 0) depends on head h (store 0) -> no head overlap
# GQA: K/V tiles loaded + transposed ONCE, reused by every query head i
11 schedule:
12
13 - id: 0
14   op: dma_load
15   unit: dma
16   label: "[KV preload] load_K_tile[j=0] (shared by 4 q-heads)"
17   params:
18     source: "HBM.K[kv0,0:128,0:128]"
19     destination: "shared_ibuf.K_tile.j0"
20     rows: 128
21     cols: 128
22
23 - id: 1
24   op: transpose
25   unit: access_core
26   label: "[KV preload] transpose_K_tile[j=0] -> K_T[j=0]"
27   depends_on: [0]
28   params:
29     source: "shared_ibuf.K_tile.j0"
30     destination: "shared_ibuf.K_T.j0"
31     input_rows: 128
32     input_cols: 128
33     output_rows: 128
34     output_cols: 128
35
36 - id: 2
37   op: dma_load
38   unit: dma
39   label: "[KV preload] load_V_tile[j=0] (shared by 4 q-heads)"
40   depends_on: [1]
41   params:
42     source: "HBM.V[kv0,0:128,0:128]"
43     destination: "shared_ibuf.V_tile.j0"
44     rows: 128
45     cols: 128
46
47 - id: 3
48   op: dma_load
49   unit: dma
50   label: "[KV preload] load_K_tile[j=1] (shared by 4 q-heads)"
51   depends_on: [2]
52   params:
53     source: "HBM.K[kv0,128:256,0:128]"
54     destination: "shared_ibuf.K_tile.j1"
55     rows: 128
56     cols: 128
57
58 - id: 4
59   op: transpose
60   unit: access_core
61   label: "[KV preload] transpose_K_tile[j=1] -> K_T[j=1]"
62   depends_on: [3]
63   params:

```

```

# units; no instance is hand-assigned. GQA: K/V loaded + transposed ONCE
11
12
13 schedule:
14
15 - id: 0
16   op: dma_load
17   unit: dma
18   label: "[KV preload] load_K_tile[j=0] (shared by 4 q-heads)"
19   params:
20     source: "HBM.K[kv0,0:128,0:128]"
21     destination: "shared_ibuf.K_tile.j0"
22     rows: 128
23     cols: 128
24
25 - id: 1
26   op: transpose
27   unit: access_core
28   label: "[KV preload] transpose_K_tile[j=0] -> K_T[j=0]"
29   depends_on: [0]
30   params:
31     source: "shared_ibuf.K_tile.j0"
32     destination: "shared_ibuf.K_T.j0"
33     input_rows: 128
34     input_cols: 128
35     output_rows: 128
36     output_cols: 128
37
38 - id: 2
39   op: dma_load
40   unit: dma
41   label: "[KV preload] load_V_tile[j=0] (shared by 4 q-heads)"
42   params:
43     source: "HBM.V[kv0,0:128,0:128]"
44     destination: "shared_ibuf.V_tile.j0"
45     rows: 128
46     cols: 128
47
48 - id: 3
49   op: dma_load
50   unit: dma
51   label: "[KV preload] load_K_tile[j=1] (shared by 4 q-heads)"
52   params:
53     source: "HBM.K[kv0,128:256,0:128]"
54     destination: "shared_ibuf.K_tile.j1"
55     rows: 128
56     cols: 128
57
58 - id: 4
59   op: transpose
60   unit: access_core
61   label: "[KV preload] transpose_K_tile[j=1] -> K_T[j=1]"
62   depends_on: [3]
63   params:
64     source: "shared_ibuf.K_tile.j1"
65     destination: "shared_ibuf.K_T.j1"
66     input_rows: 128
67     input_cols: 128
68     output_rows: 128
69     output_cols: 128
70
71 - id: 5
72   op: dma_load
73   unit: dma
74   label: "[KV preload] load_V_tile[j=1] (shared by 4 q-heads)"
75   params:

```

Handwritten/Generated Static Schedule for GQA_FA2 Attention - Single & Overlapped

Reserving from Unit Pool

```
23
24 systolic_units: 2
25 vector_cores: 3
26 access_cores: 1
27
28 ▼ sram:
29   ibuf_kb: 4096 # shared input buffer
30   obuf_kb: 4096 # shared output buffer
31   banking_factor: 8 # concurrent r/w ports per cycle
32   private_vector_kb: 512 # per-vector-core private SRAM
33
34 ▼ hbm:
35   bandwidth_tb_s: 2.0 # H100 ~3.35 TB/s; TPUv4 ~1.2 TB/s
36   latency_cycles: 200
37   pipelined: true # P1.5: channel occupancy = bandwidth term only
38   # (latency overlaps adjacent transfers)
39
40 ▼ dma:
41   channels: 1
42
43 ▼ vector_core:
44   simd_width: 64 Nikhita, 2 weeks ago • all hw units
45   exp_latency: 4
46
47 ▼ access_core:
48   bandwidth: 64
```

```
auto res = ctx.scheduler.reserve_unit_pool(targets, lat);
Event e;
e.type = EventType::OP_START;
e.target = res.id;
e.cycle = res.start;
e.instr = ctx.inst.id;
e.label = ctx.inst.label;
e.payload = op;
ctx.engine.schedule(std::move(e));
```

- Reserve whatever units free instead of by hand or generating links from events to available units
- Aids in maximally utilizing units
- Enables architecture studies

	Per Tile		Per Element	Per Row Sub-Tile	Per Element	Per Tile
Systolic Array			$S_tile = Q_tile \times K_tile_T$			$Temp = P_tile \times V_tile$
Tandem-Stream			$S_tile = S_tile / \sqrt{128}$		$O_acc[i, :] = O_acc[i, :] * correction[i]$	$O_acc = O_acc + Temp$
Tandem					$P_tile[i][j] = \exp(S_tile[i][j] - m_new[i])$	
Access Core		$K_tile_T = \text{transpose}(K_tile)$	$row_max(S_tile)$	$m_new[i] = \max(m[i], m_tile[i])$ $correction[i] = \exp(m[i] - m_new[i])$ $[i] = [i] * correction[i]$	$[i] = [i] + \text{sum over } j: P_tile[i][j]$	
DMA	Load K_tile	Load V_tile				

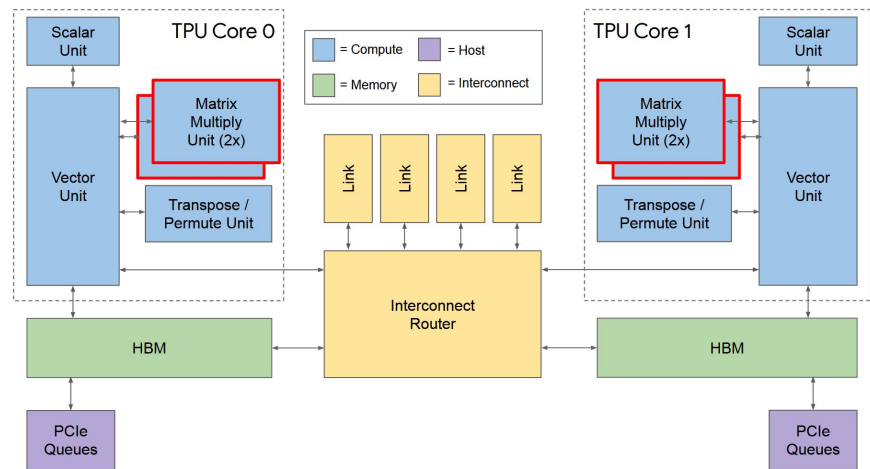
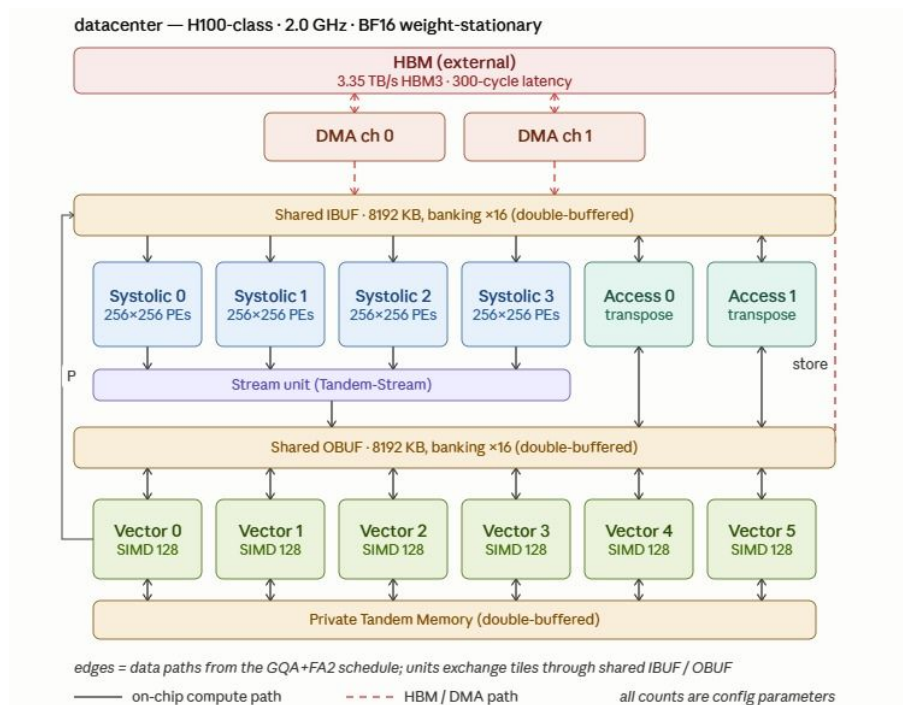


Systolic Array			$S_tile = Q_tile \times K_tile_T$		$S_tile = Q_tile \times K_tile_T$	$Temp = P_tile \times V_tile$	$S_tile = Q_tile \times K_tile_T$	$Temp = P_tile \times V_tile$	$S_tile = Q_tile \times K_tile_T$	$Temp = P_tile \times V_tile$
Tandem 1			$S_tile = S_tile / \sqrt{128}$		$P_tile[i][j] = \exp(S_tile[i][j] - m_new[i])$		$O_acc[i, :] = O_acc[i, :] * correction[i]$	$O_acc = O_acc + Temp$	$S_tile = S_tile / \sqrt{128}$	$O_acc = O_acc + Temp$
Tandem 2					$O_acc[i, :] = O_acc[i, :] * correction[i]$	$O_acc = O_acc + Temp$	$S_tile = S_tile / \sqrt{128}$		$O_acc[i, :] = O_acc[i, :] * correction[i]$	
Tandem 3					$S_tile = S_tile / \sqrt{128}$		$P_tile[i][j] = \exp(S_tile[i][j] - m_new[i])$		$P_tile[i][j] = \exp(S_tile[i][j] - m_new[i])$	
Access Core 1		$K_tile_T = \text{transpose}(K_tile)$	$row_max(S_tile)$	$m_new[i] = \max(m[i], m_tile[i])$ $correction[i] = \exp(m[i] - m_new[i])$ $[i] = [i] * correction[i]$	$[i] = [i] + \text{sum over } j: P_tile[i][j]$	$m_new[i] = \max(m[i], m_tile[i])$ $correction[i] = \exp(m[i] - m_new[i])$ $[i] = [i] * correction[i]$	$[i] = [i] + \text{sum over } j: P_tile[i][j]$	$m_new[i] = \max(m[i], m_tile[i])$ $correction[i] = \exp(m[i] - m_new[i])$ $[i] = [i] * correction[i]$	$[i] = [i] + \text{sum over } j: P_tile[i][j]$	$m_new[i] = \max(m[i], m_tile[i])$ $correction[i] = \exp(m[i] - m_new[i])$ $[i] = [i] * correction[i]$
Access Core 2				$K_tile_T = \text{transpose}(K_tile)$	$row_max(S_tile)$	$K_tile_T = \text{transpose}(K_tile)$	$row_max(S_tile)$	$K_tile_T = \text{transpose}(K_tile)$	$row_max(S_tile)$	$K_tile_T = \text{transpose}(K_tile)$
DMA	Load K_tile	Load V_tile	Load K_tile	Load V_tile	Load K_tile	Load V_tile	Load K_tile	Load V_tile	Load K_tile	Load V_tile

Flexibility

- Can register new operations available to scheduler, reserved on predefined hardware unit classes through `op_handlers.cpp`
- New hardware units build under `units/` -> Example if you want to add some sort of sparse unit
 - Need `set_scheduler` and `handle` (compute latency, dependencies, etc.) methods
 - Call on unit from passed schedule (DAG)
 - Ensure unit is registered using `engine.register_unit()` under `sim_main.cpp`

Configurable Hardware Configurations - Datacenter



[Google TPUv3](#)

Stylized Diagram of Config

```
clock_ghz: 2.0

systolic:
  rows: 256
  cols: 256
  precision: BF16
  bidirectional: false
  d_head: 128
  dataflow: weight_stationary
  weight_load_cycles: 0
  weight_double_buffer: true

structural_k_tiling: false
model_sram: true
stage_double_buffer: true

systolic_units: 4 # try with 8 - TPUv1 thru TPUv5
vector_cores: 6
access_cores: 2

sram:
  ibuf_kb: 8192
  obuf_kb: 8192
  banking_factor: 16
  private_vector_kb: 1024

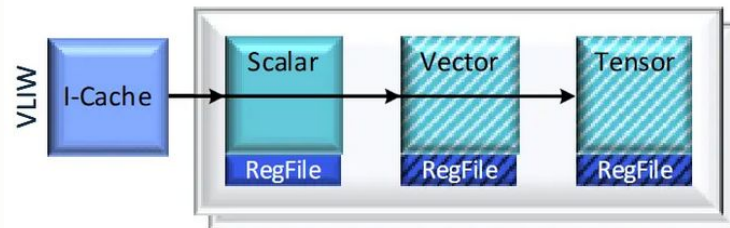
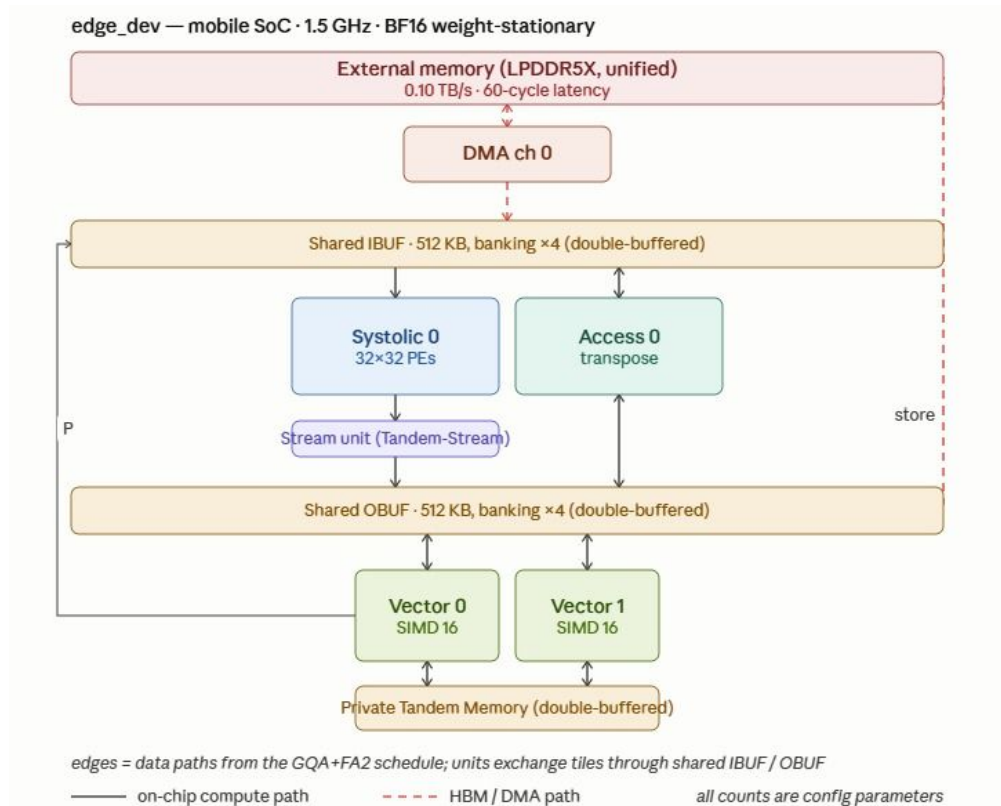
hbm:
  bandwidth_tb_s: 3.35 # H100 HBM3
  latency_cycles: 300 # ~150ns x 2 GHz
  pipelined: true

dma:
  channels: 2

vector_core:
  simd_width: 128
  exp_latency: 4

access_core:
  bandwidth: 128
```

Configurable Hardware Configurations - Edge



[Snapdragon 8 Gen 2, Hexagon NPU](#)

Stylized Diagram of Config

```

2  # Models an Apple M-class or Snapdragon-class SoC:
3  # - LPDDR5X unified memory (~77-136 GB/s = 0.077-0.136 TB/s)
4  # - Small systolic array (16x16 or 32x32 NPU tile)
5  # - Higher clock, much lower memory bandwidth than HBM
6  # Reference: Apple M3 (memory BW ~150 GB/s), Qualcomm NPU (LPDDR5X ~77 GB/s)
7
8  clock_ghz: 1.5
9
10 systolic:
11     rows: 32
12     cols: 32
13     precision: BF16
14     bidirectional: false
15     d_head: 128
16     dataflow: weight_stationary
17     weight_load_cycles: 0
18     weight_double_buffer: true
19
20 structural_k_tiling: false
21 model_sram: true
22 stage_double_buffer: true
23
24 systolic_units: 1
25 vector_cores: 2
26 access_cores: 1
27
28 sram:
29     ibuf_kb: 512
30     obuf_kb: 512
31     banking_factor: 4
32     private_vector_kb: 128
33
34 hbm:
35     bandwidth_tb_s: 0.10 # LPDDR5X ~100 GB/s unified
36     latency_cycles: 60 # ~40ns × 1.5 GHz (LPDDR is lower latency than HBM)
37     pipelined: true
38
39 dma:
40     channels: 1
41
42 vector_core:
43     simd_width: 16
44     exp_latency: 8
45
46 access_core:
47     bandwidth: 16

```

Hardware Unit Calculations

- **Systolic weight-stationary GEMM**
 - $\text{tiles_k} = \lceil K / \text{rows} \rceil$, $\text{tiles_n} = \lceil N / \text{cols} \rceil$, $\text{fill} = (\text{rows} - 1) + (\text{cols} - 1)$. Double-buffered: $\text{lat} = \text{tiles_n} \cdot (\text{wload} + \text{tiles_k} \cdot \max(\text{wload}, M) + \text{fill})$.
- **DMA - HBM retrieval - +intrinsic hbm lat**
 - $\text{data ready} = \text{hbm_lat} + \lceil \text{bytes} / (\text{bw} \cdot \text{channels}) \rceil$.
 - Stage (on-chip sram): $\lceil \text{elements} / \text{rows} \rceil$ over the wide operand bus.
- **Vector -> passes = sweeps over data per op**
 - $\text{lat} = \text{passes} \cdot \lceil \text{elem} / \text{simd} \rceil + \text{exp_ops} \cdot \text{exp_lat} \cdot \lceil \text{elem} / \text{simd} \rceil$.
 - $\text{lat} = \text{cycles to pass into lanes} + \text{stages per op} * \text{latency} * \text{num_groups}$
- **Access**
 - $\text{transpose / copy / init: } \lceil \text{elements} / \text{bandwidth} \rceil$

Number of element in tensor passed in - elems

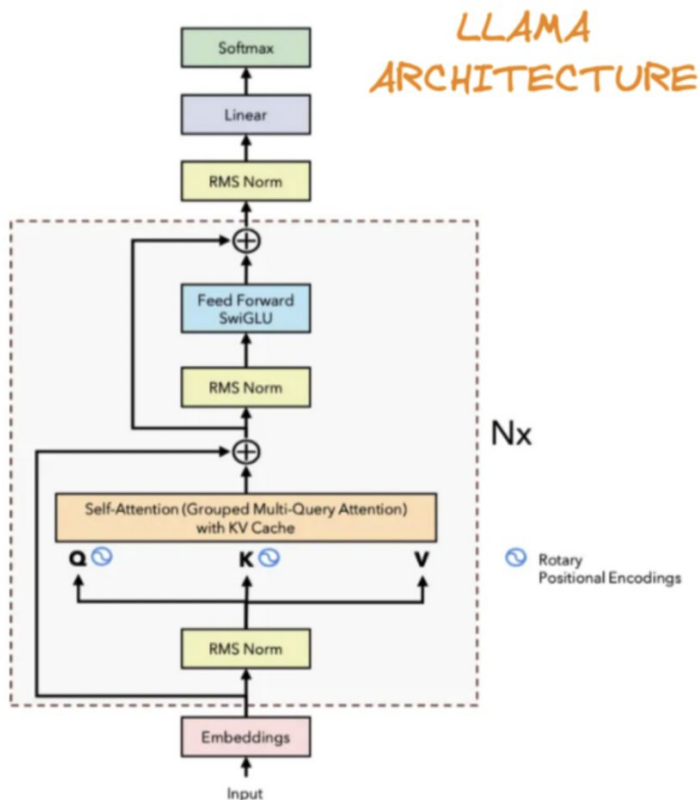
The simulated workload

LLaMA3b

Schedule Details

- There is no handwritten schedule. Instead we have:
 - A workload config which defines the model params to run.
 - Cpp schedule generator which expands prefill and decode into granular steps.
 - Runtime op handlers and hw units which interpret these generated instructions on hw units and assign latency estimates.
 - Units: DMA, Access cores, Vector units, systolic array; memory devices: HBM, SRAM
- At runtime, a schedule is created consisting of instruction objects
 - Each one contains an ID, op name, target unit, numeric params (matmul size), dependency ids.
 - Then we expand tiling events into subtiling events depending on whether a matmul will fit on the systolic array.
- After this instruction graph is complete, the event engine creates runtime events calculates latency.

LLaMA



- Simulating LLaMA via an event-driven simulator requires breaking each of these blocks into detailed events that depend on each other.
- This architecture is pretty standard for most LLMs. You embed then perform attention, then FFN, softmax, plus some skip connections.
- We started off with building a schedule for the Attention block (This contains everything after embedding, to the output of the attention mechanism).
- The LLM schedule implements FlashAttention2 and Grouped Query Attention, both methods that reduce the memory load on the system.

Flash Attention 2

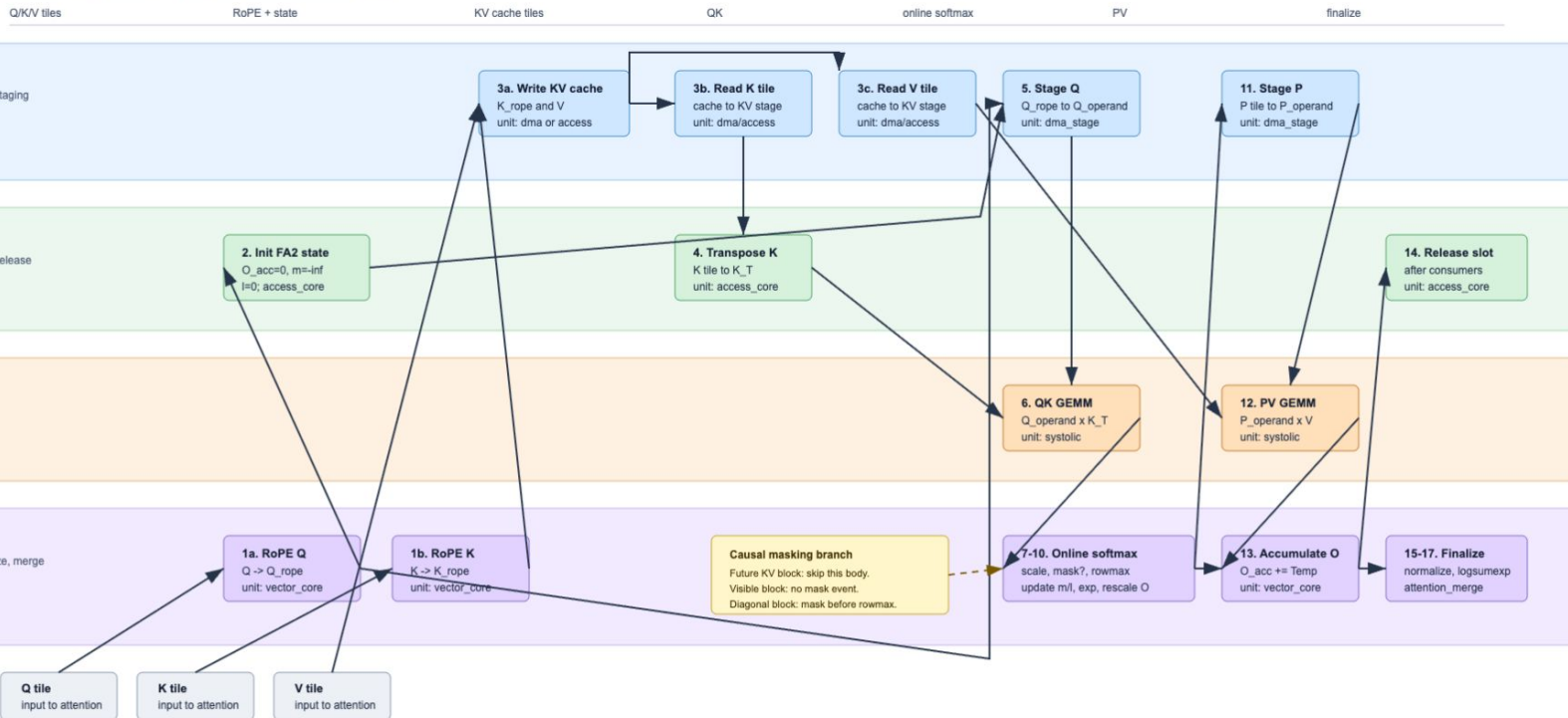
- FA2 starts with loading K, V, and Q tiles from HBM→SRAM using the DMA. DMA also gets used to send Q tile from SRAM to systolic array
- Concurrently, transpose K tile, and initialize m,l vectors, and output matrix via Access core. The access core feeds K to the systolic array.
- Systolic array is used for the QK^T GEMM.
- scaling , rowmax, exponentiation, vector additions, accumulates are all done by the vector units.
- Note that $P = \text{softmax}(QK^T)$. P is used to update l vector. This is a vec unit operation.
- P then gets loaded into systolic array via DMA. V is fed into systolic array
- PV GEMM is done on systolic array
- Output gets accumulated via vector unit

Things to note:

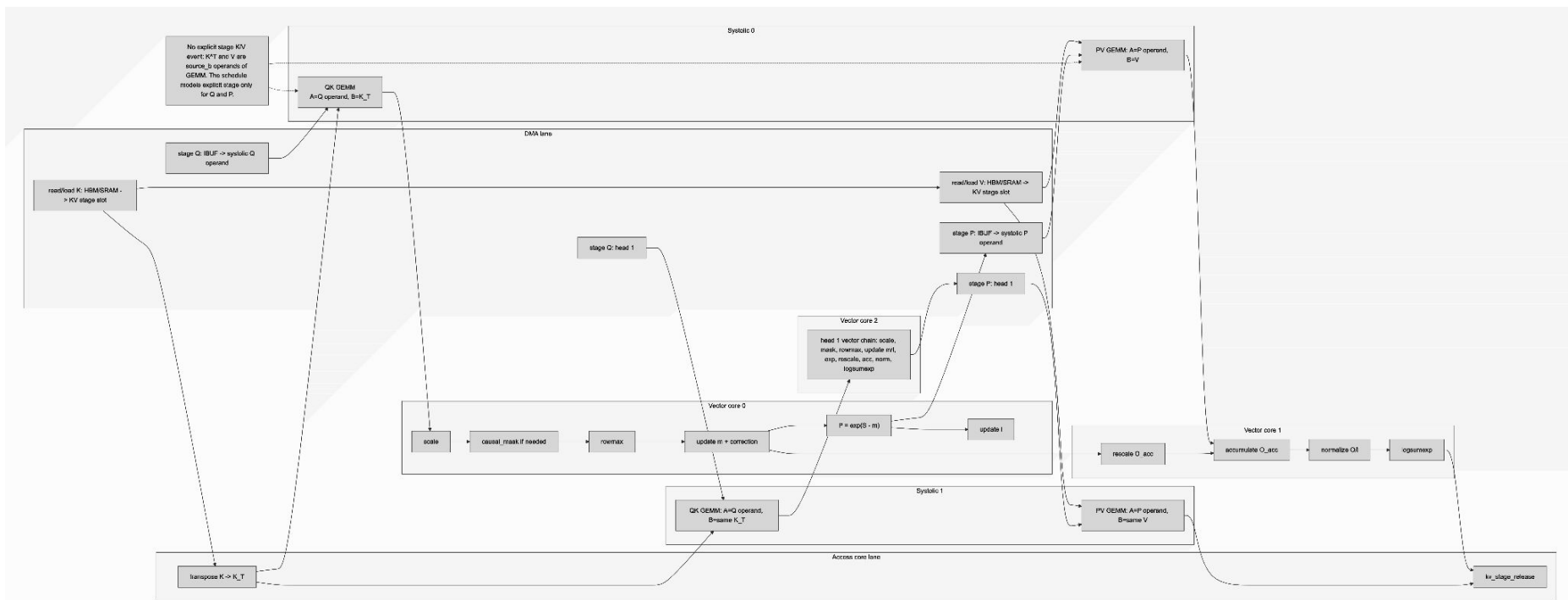
- Although FlashAttention tiles Q,K,V matrices into tiles, these tile sizes are not chosen based on the systolic array size alone. This means, if the tiles are larger than the systolic array, a single tiled event gets further tiled. However we don't model this as a hardware process; rather we subtile into separate systolic array events.
- In attention, we apply a causal mask such that tokens can only attend to those that came before them. In QK^T , this means if we compute the whole matrix, half of it gets set to -inf because those values are not used in the calculation.
 - As such, the scheduler handles this by checking whether a KV tile is going to get completely masked, in which case we can avoid scheduling that tile. Otherwise, if it is partially masked, we can insert the mask. If the KV tile is fully visible, then we will let it pass thru unaffected.
 - This cuts down unnecessary computations by half.
 - llama_schedule.cpp (line 1030)

Attention tile schedule, arranged by hardware unit

Q, K, and V tiles are the starting point. Projection weights and Q/K/V projection GEMMs are out of scope.



The dependency graph for FA2



Additional details

- Schedule includes events for positional encoding (RoPE) which occurs after Q,K,V tiles are loaded.
- KV Cache write events occur pushing KV vectors to HBM.
- Then GQA manages the way attention occurs for kv groups.
 - For each Q tile and KV tile, FA2 sequence occurs, containing online softmax, among other details.
 - Note that Q tile stays on the systolic array as K and V tiles pass through.

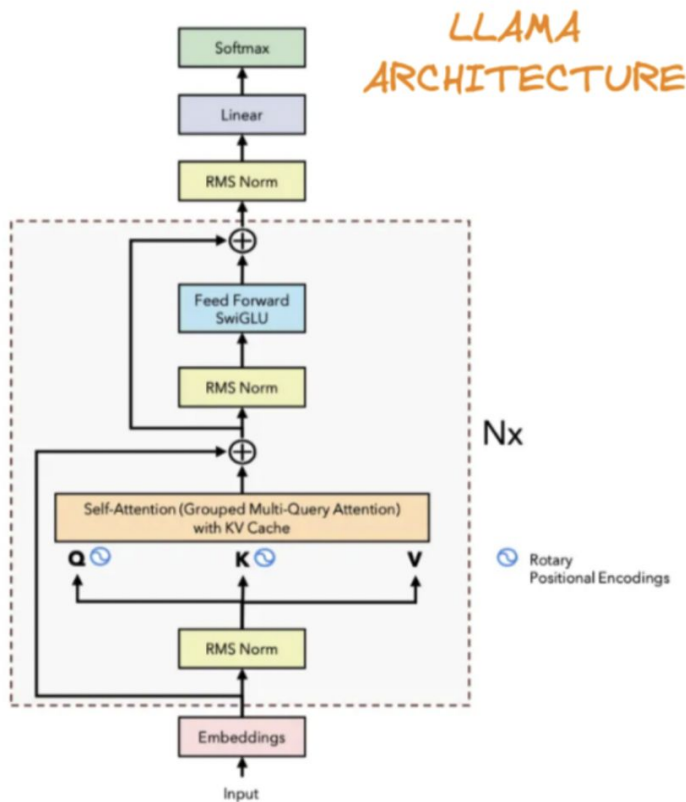
KV Cache & Prefill-Decode

- An LLM workload can be split into 2 separate portions: prefill and decode
- In Prefill:
 - the initial input to the model is processed all at once, requiring a lot of compute
 - For each input token, k and v vectors are extracted and used to calculate the QK^T output.
 - These will be needed again for generation so they are saved to HBM.
- Decode:
 - Starts at the generation of the first output token. This token's k and v vectors are appended to the KV Cache.
 - Note that during decode, the future tokens attend to the previous ones. As such, a future token only requires the prior key vectors, not the query.
 - This is memory heavy, and kv cache grows linearly as tokens are generated.
- K, V vectors move from Systolic Array $\rightarrow$ SRAM $\rightarrow$ HBM, and can be pulled from there when needed.

In the generated schedule:

- K and V are first produced by projection:
llama_schedule.cpp (line 875)
- Q and K get RoPE:
llama_schedule.cpp (line 885)
- K/V are written into the KV cache when kv_cache_enabled is true:
llama_schedule.cpp (line 899)
- Later, each KV tile is read back before attention:
llama_schedule.cpp (line 980)
- After all query-head consumers finish using that staged K/V tile, the staging slot is released:
llama_schedule.cpp (line 1119)

Interlude for the rest of the model



- From a single tile of attention, we scaled a schedule for a full attention output.
- Then we developed schedules for the rest of the model:
 - Before Attention, you have embedding and positional encoding - RoPE
 - After Attention, there is the RMS Norm, SWiGLU, MLP, and softmax.
 - These were all relatively simple to develop schedules for.
- Note that along the way, we shifted to schedule templates which were used to generate schedules prior to simulation.

GQA - Grouped Query Attention

- Once the model was built for a single attention block, this schedule was expanded to multi-head attention.
- GQA is used in production models because it greatly reduces the size of the KV cache. Otherwise, each block, in each layer, needs its own KV Cache.
- Implementing GQA meant having multiple attention blocks, grouped to share KV Cache (this is why everything else had to be built out before implementing this)
- Simulator parametrizes the number of groups and number of heads (along with many other params such as tile size for FA2)

How does the introduction of GQA change the schedule?

Within one GQA group:

load/read K,V once, transpose K once, run FA2 work for 4 Q heads using shared K_T and V.

Across different GQA groups:

K,V are different, cache reads/transposes are separate, though work can still overlap if hardware resources are free.

In the generated schedule, GQA appears in the loop structure:

- Outer loop over KV heads:
llama_schedule.cpp
(line 924)
- Inner loop over query heads local to that KV head:
llama_schedule.cpp
(line 927)

The Schedule config

```
llama:
  mode: prefill_decode
  prompt_len: 2048 # Size of the input
  generation_steps: 1 # number of tokens outputted
  num_layers: 32 # Number of Layers in the model
  num_q_heads: 32 # Query heads
  num_kv_heads: 8 # Number of KV Heads, each one a group
  gqa_group_size: 4 # Size of the KV Head group
  head_dim: 128 # Output size of each attention head. Note head_dim x num q heads = hidden dim
  hidden_dim: 4096 # Width of each token's hidden state. num q heads x head_dim
  intermediate_dim: 14336 # hidden width of the MLP (FFN) block (4096 --> 14336 --> 4096)
  vocab_size: 128256 # number of possible token ids the model can generate from
  max_seq_len: 8192 # max context size

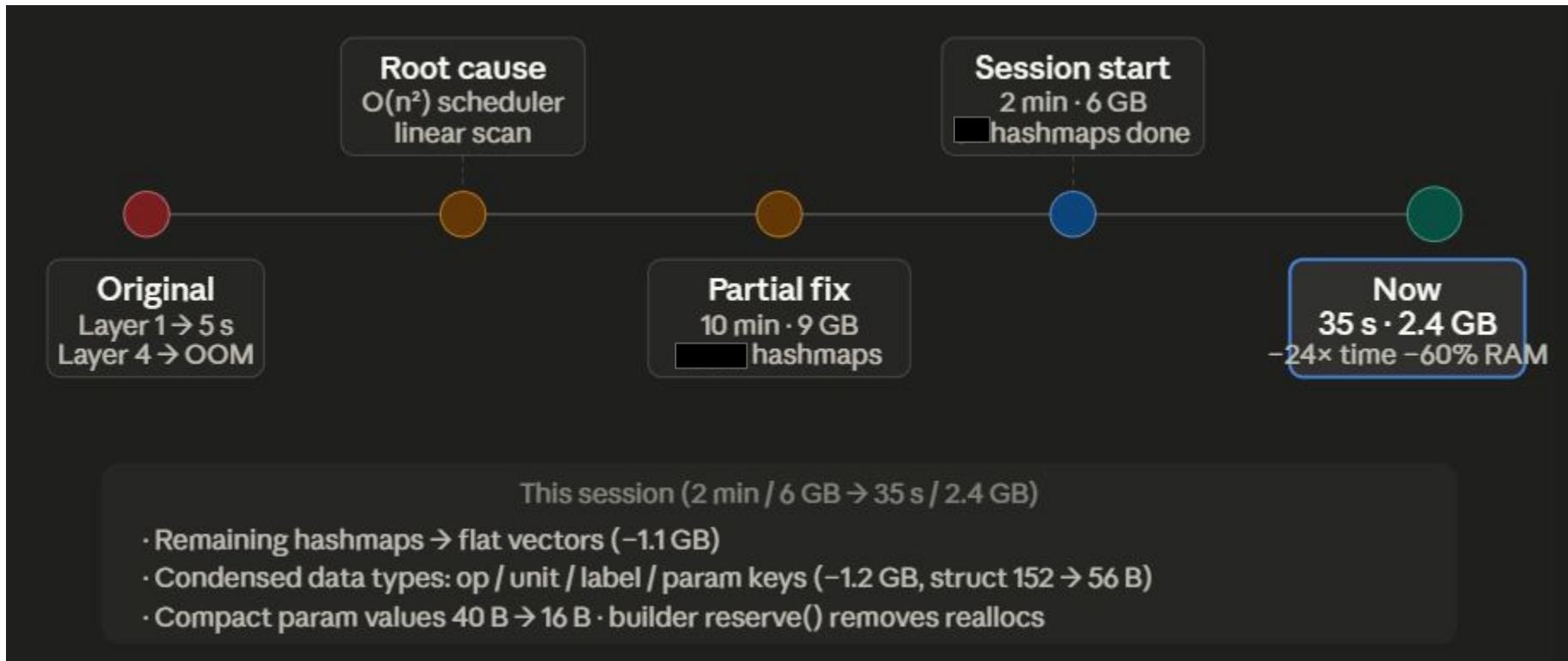
  dtype_bytes: 2 # BF16

  # Size of the Q,K,V tiling for FA2. If these dims match the dims of the systolic array, then
  # no subtiling needs to take place. However, these dims are set based on on-chip memory and other
  # hardware details. Tiles in practice tend to be larger than the systolic array.
  tile_rows: 256
  tile_cols: 256

  # If disabled, prefill would work fine but decode would recompute KV everytime
  # So effectively, KV cache is not optional.
  kv_cache_enabled: true
  kv_cache_location: hbm #location for KV Cache is usually in HBM
```

Implementation details,
Configs and Results

Performance Engineering Story



Knobs:

WORKLOAD YAML — LLAMA CONFIG

SEQUENCE

mode prefill_decode

prompt_len 2048

generation_steps 1

max_seq_len 8192

MODEL SHAPE

num_layers 32

num_q_heads 32

num_kv_heads 8

gqa_group_size 4

head_dim 128

hidden_dim 4096

intermediate_dim 14336

vocab_size 128256

TILING

tile_rows 256

tile_cols 256

KV CACHE

kv_cache_enabled true

kv_cache_location hbm

PRECISION

dtype_bytes 2 (BF16)

HARDWARE CONFIG — DEFAULT.YAML

CLOCK

clock_ghz 1.0

SYSTOLIC / MXU

systolic_units 2

rows / cols 256 × 256

precision BF16

d_head 128

dataflow weight_stationary

weight_load_cycles 0 (auto)

weight_double_buffer true

bidirectional true

SRAM MODELING

structural_k_tiling false

model_sram true

stage_double_buffer true

SRAM

ibuf_kb 4096

obuf_kb 4096

banking_factor 8

private_vector_kb 512

HARDWARE CONFIG — DEFAULT.YAML

HBM

bandwidth_tb_s 2.0

latency_cycles 200

pipelined true

DMA / VECTOR / ACCESS

dma.channels 1

vector_cores 3

simd_width 64

exp_latency 4

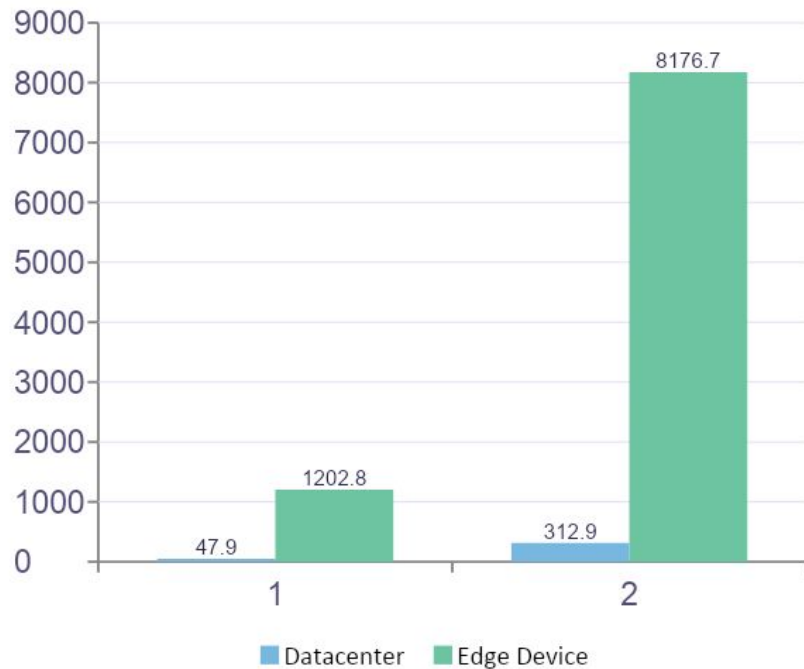
access_cores 1

access_core.bandwidth 64

Case Study :Edge devices vs Data Center - Inference

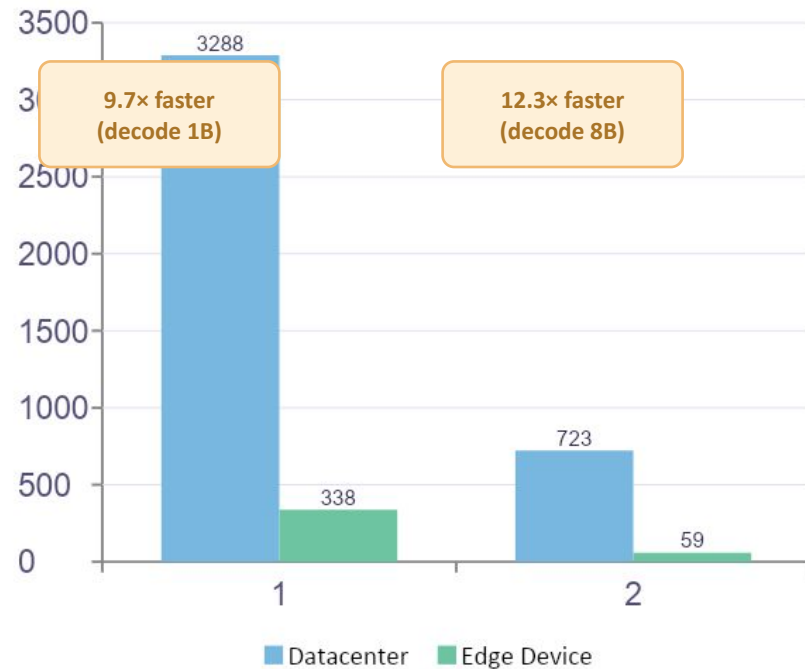
Prefill: TTFT per Input Token

lower is better · $\mu\text{s} / \text{token}$



Decode Throughput

higher is better · $\text{tokens} / \text{second}$



* 8B model: Datacenter ran 2,048-token prompt; Edge ran 512-token prompt (longer caused OOM)

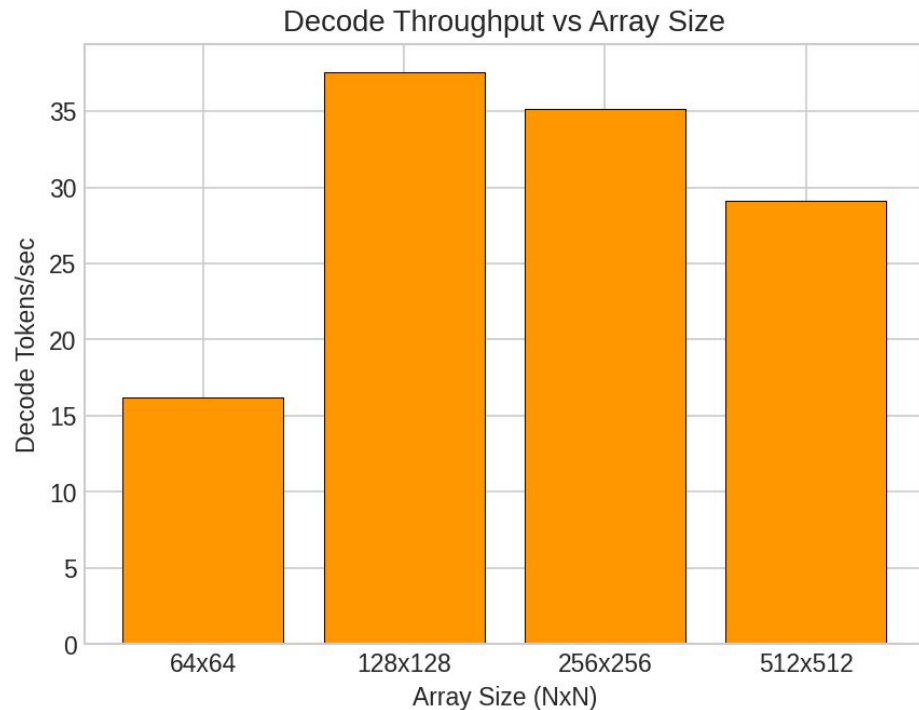
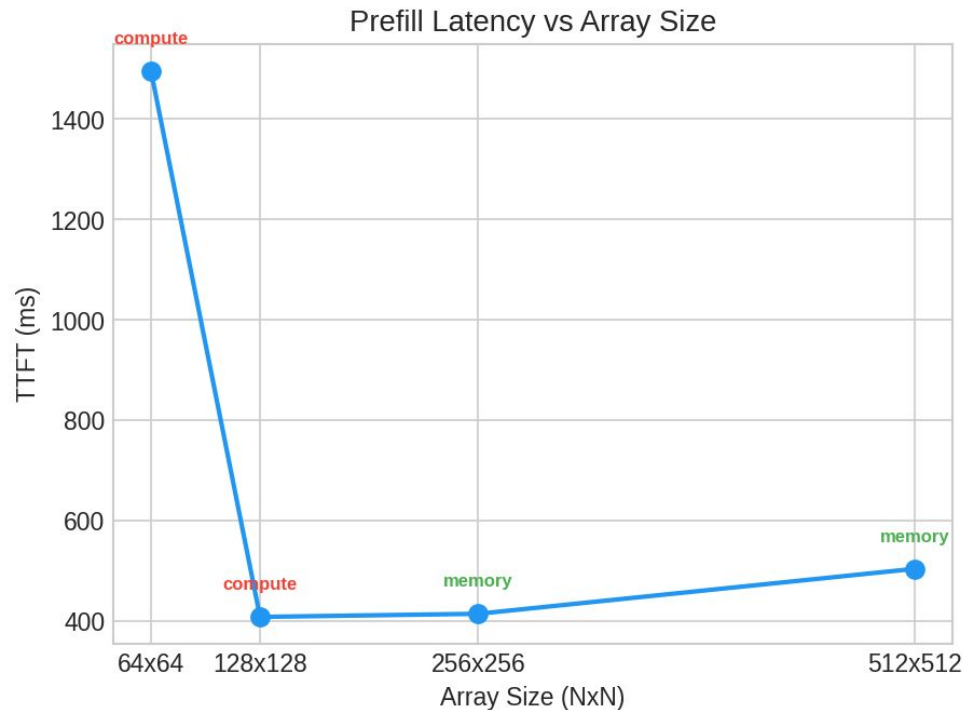
```
num_layers: 16           # was 32
num_q_heads: 32          # same
num_kv_heads: 8           # same
gqa_group_size: 4         # 32/8
head_dim: 64             # was 128
hidden_dim: 2048          # was 4096
intermediate_dim: 8192    # was 14336
```

```
2 systolic arrays
3 vector cores
1 access core
```

[illegible]

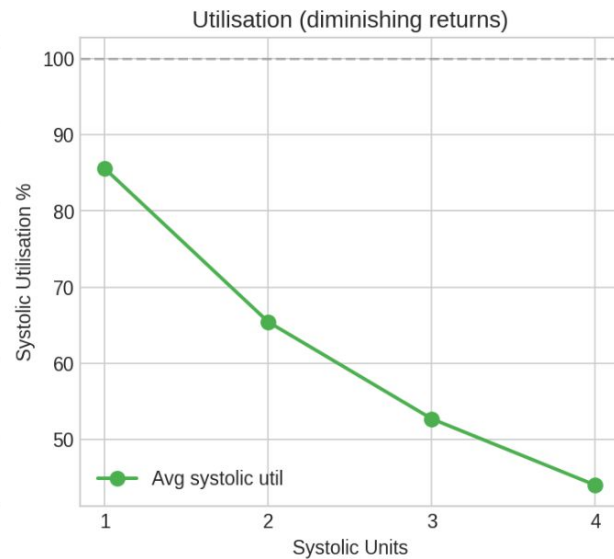
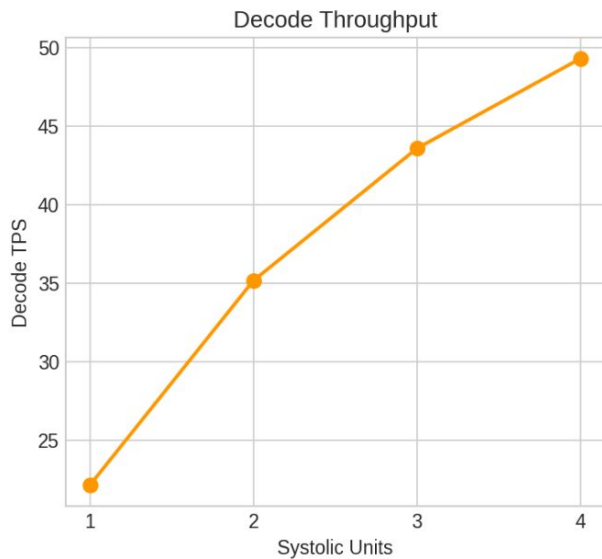
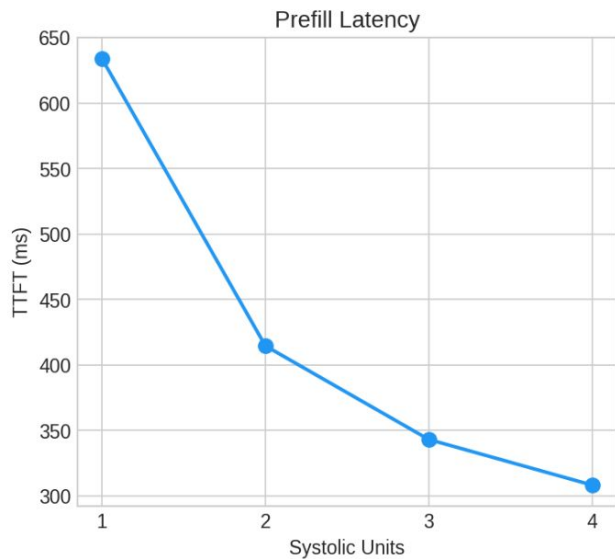
Group 1: Compute (1B model), FA2 tile size - 256, 256

1a: Array Size Scaling — Llama 1B (BF16)



Group 1: Compute (1B model)

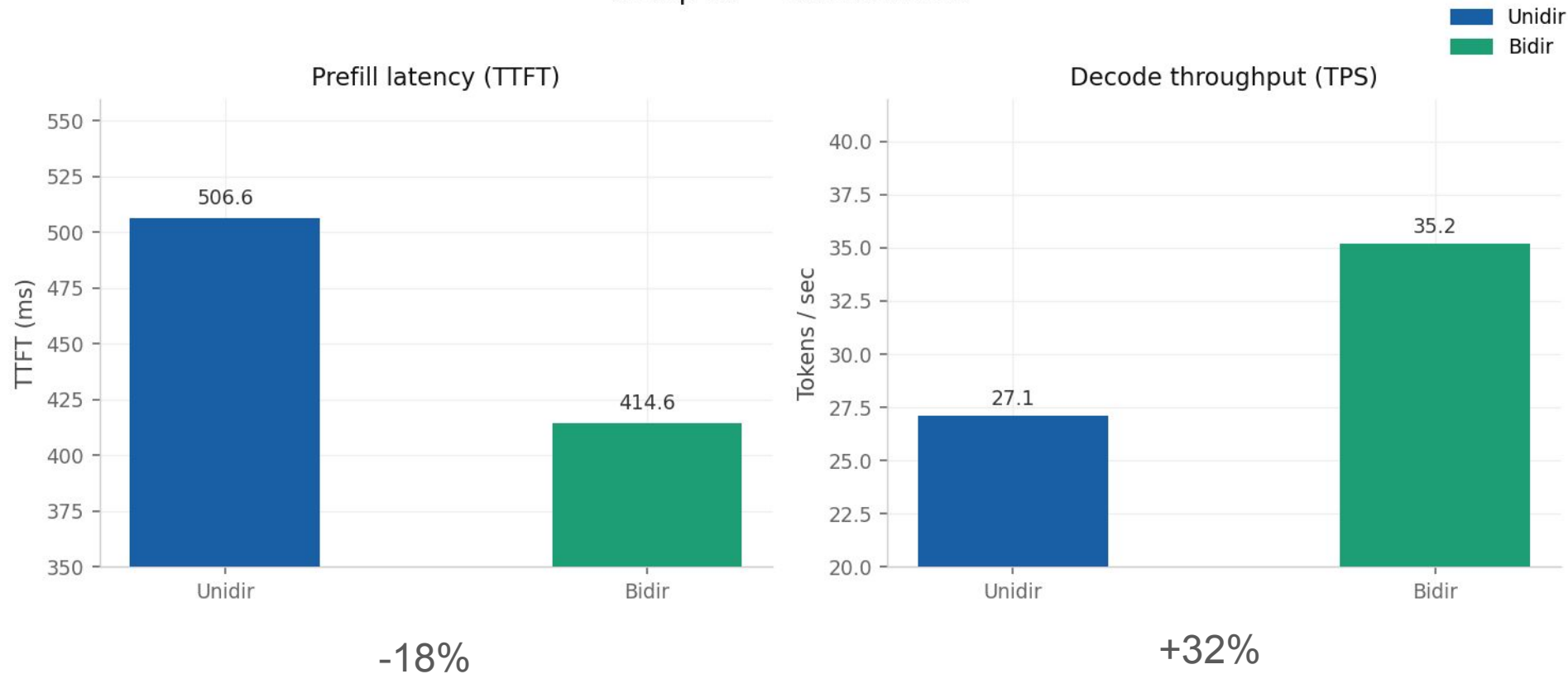
1b: Systolic Unit Scaling — Sub-linear Returns Due to Memory Bound



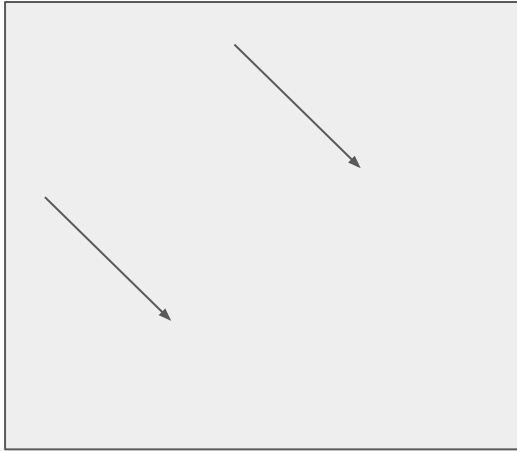
23 to 49 TPS

Group 1: Compute (1B model)

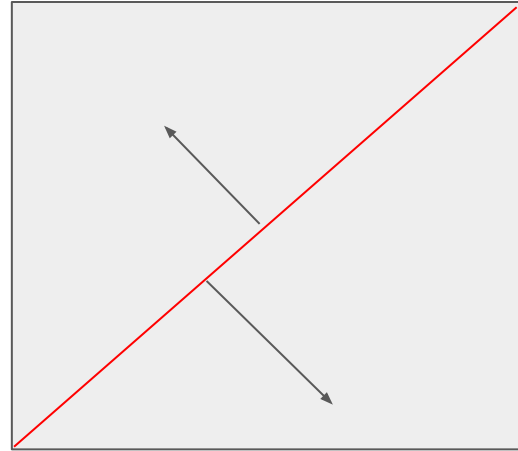
Group 1c — Bidirectional



Unidirectional flow

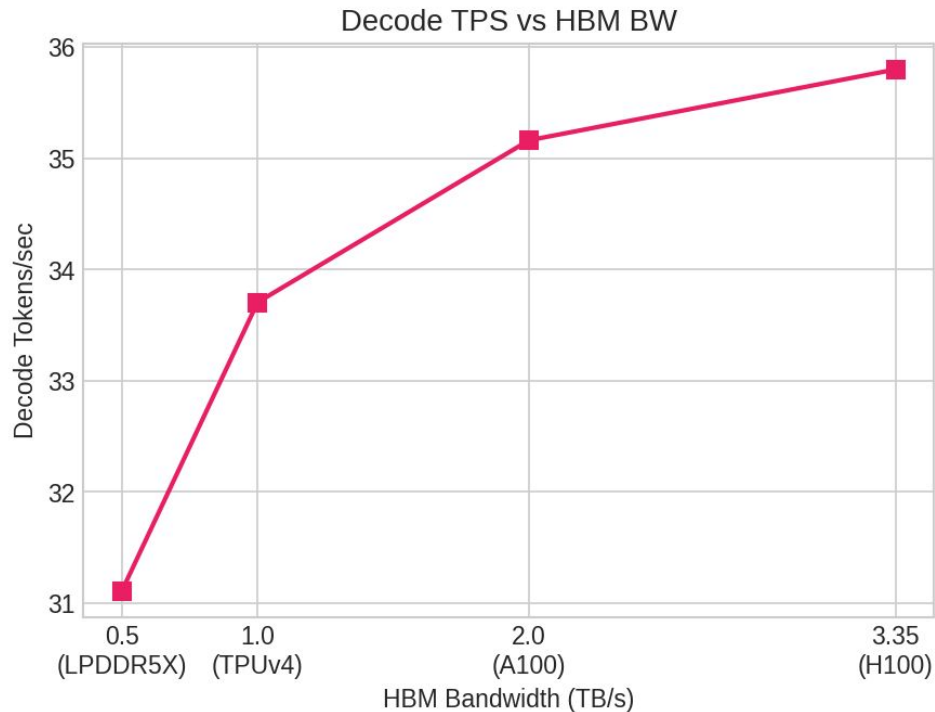
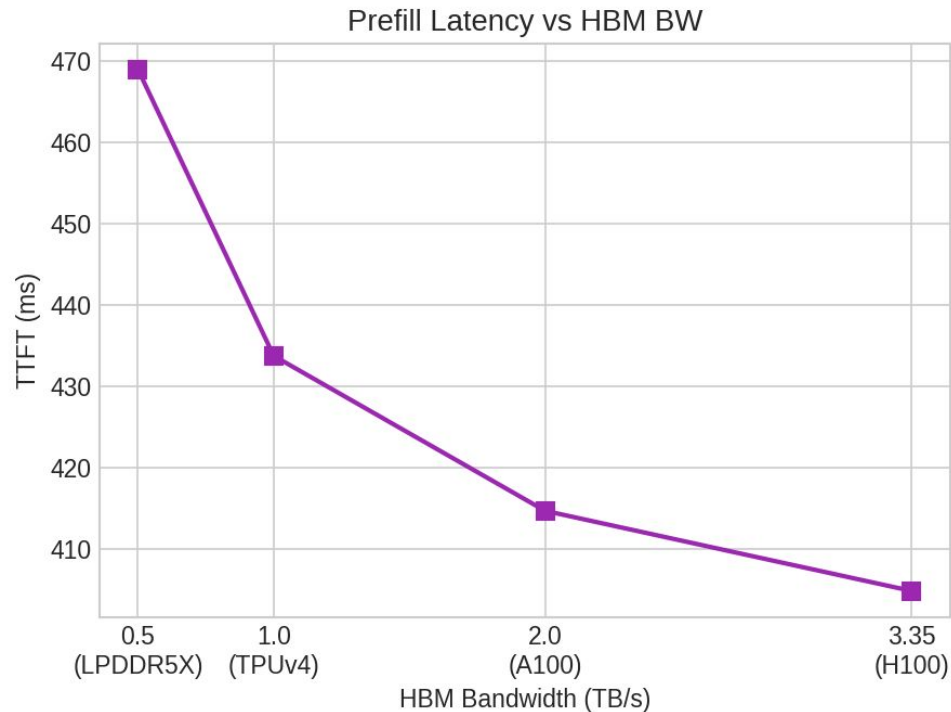


Bidirectional flow



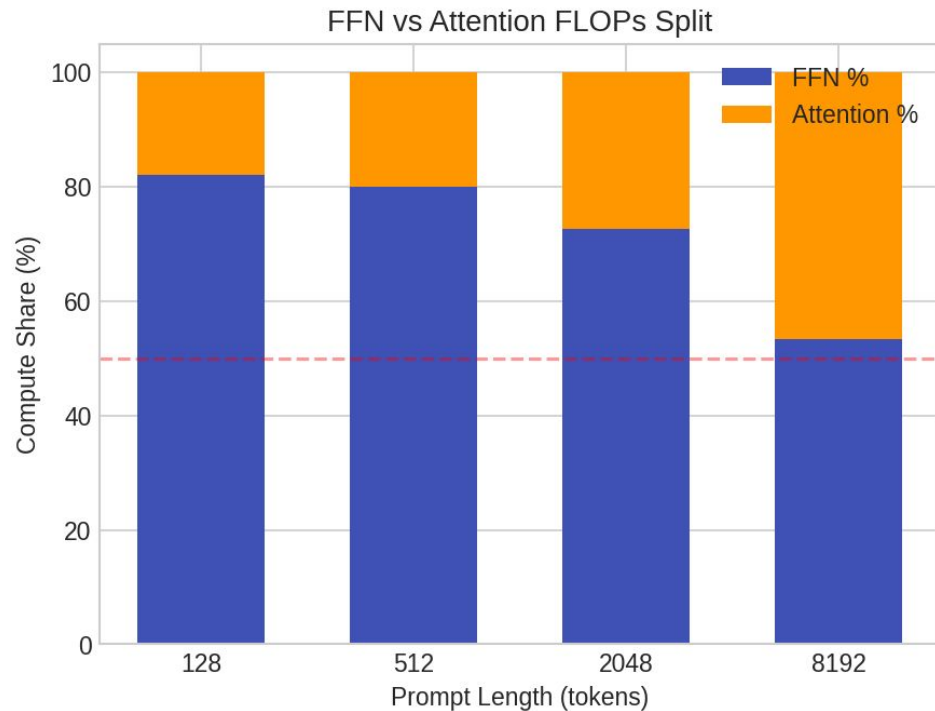
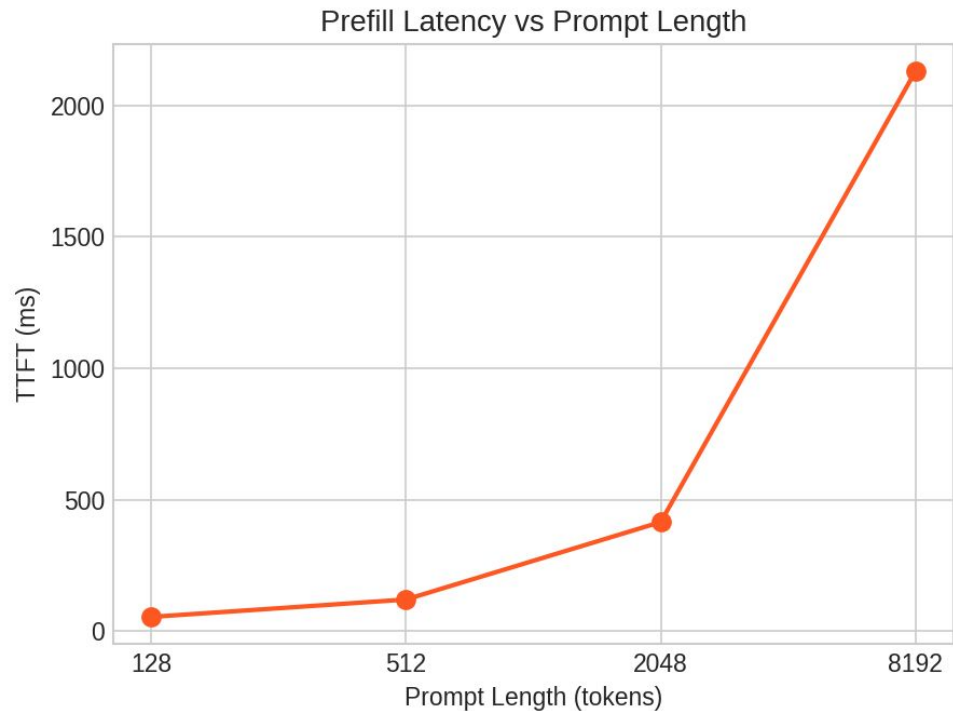
Group 2: Memory (1B model): 32 tokens on decode

2a: HBM Bandwidth — Memory Wall Dominates Decode



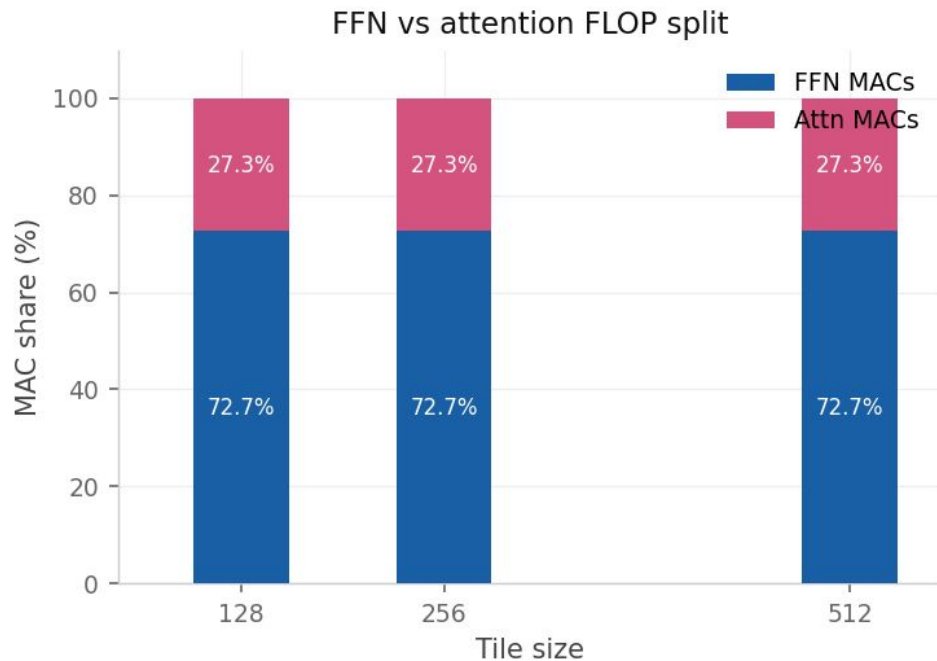
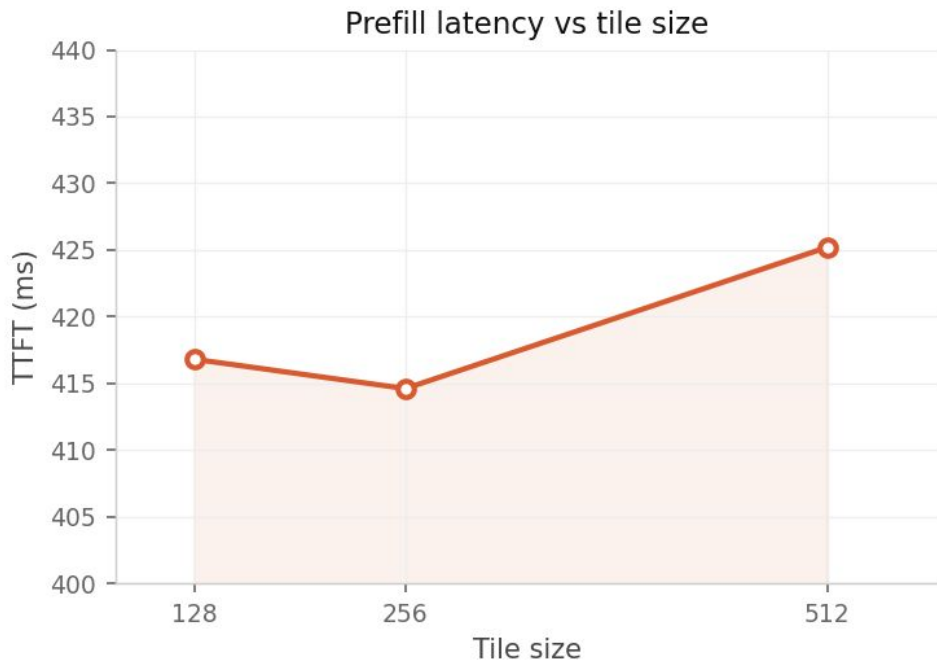
Group 3: Workload Knobs (1B Model)

3a: Prompt Length — Regime Shift from FFN-Dominated to Attention-Heavy



Group 3: Workload Knobs (1B Model), 256x 256 sys array

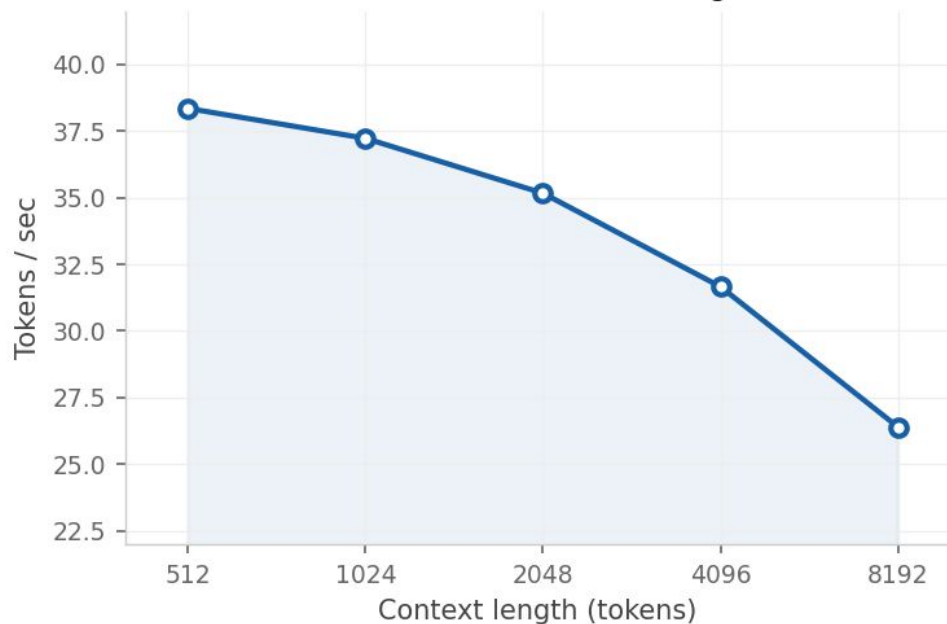
Group 3b — Tile Size (prompt = 2048 tokens)



Group 3: Workload Knobs (1B Model)

Group 3e — Decode Context Length

Decode TPS vs context length

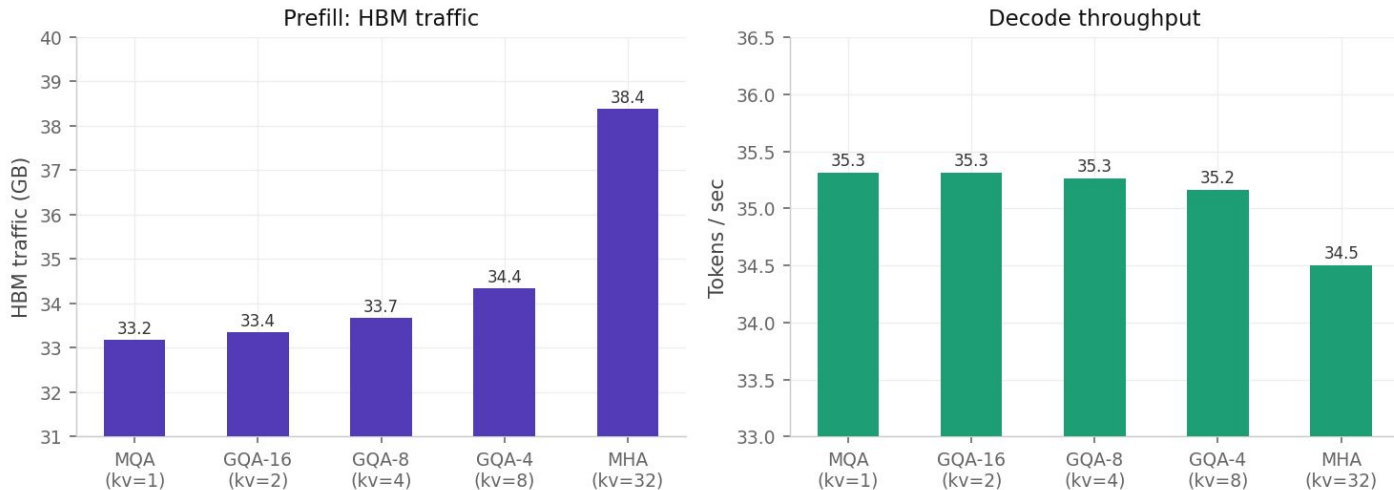


Memory footprint growth with context



Group 4 - Multi Head Attention (1B Model)

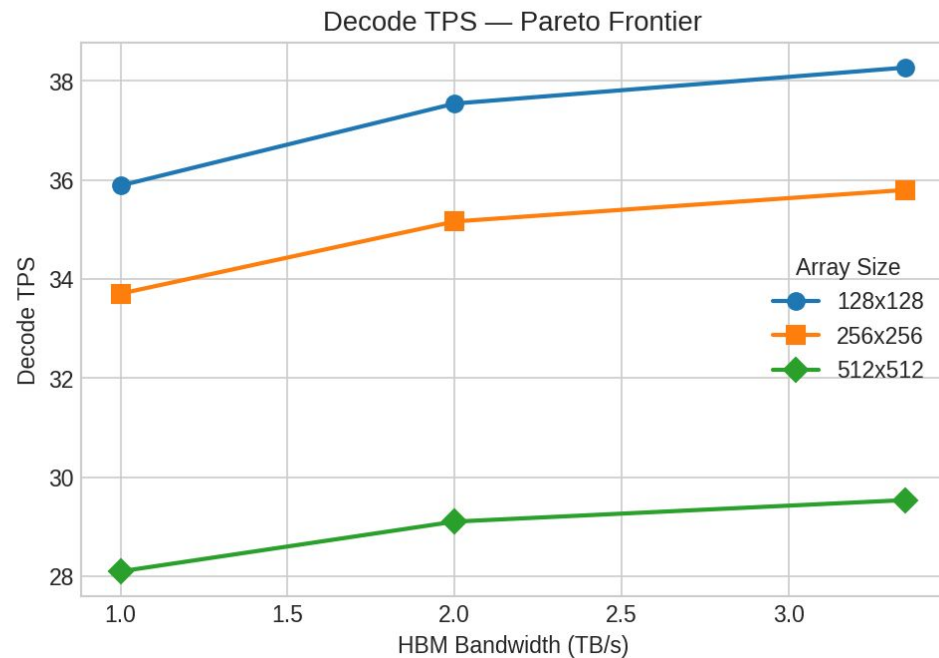
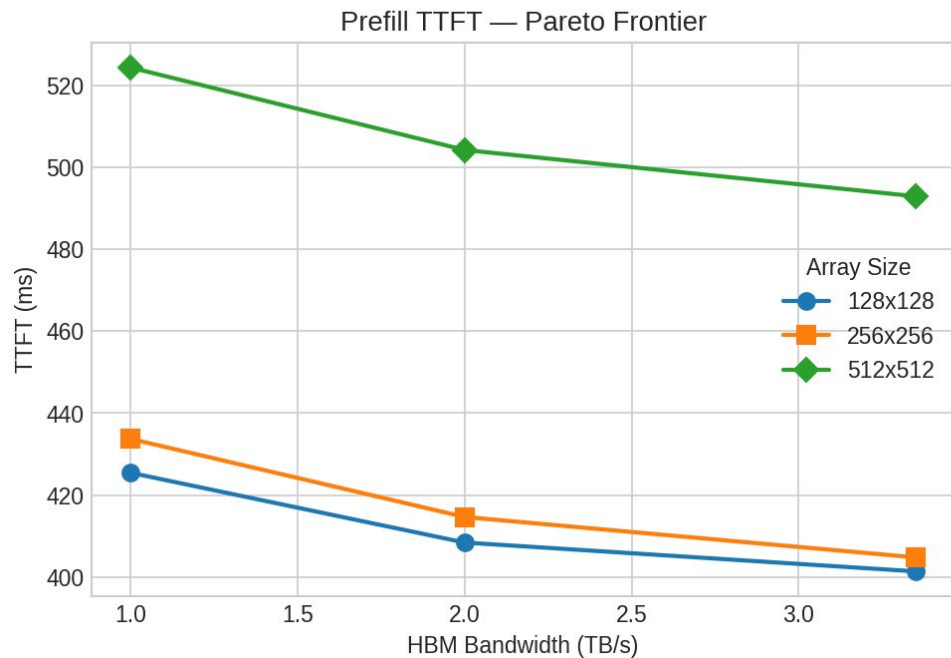
Group 4a — Attention Type (KV heads)

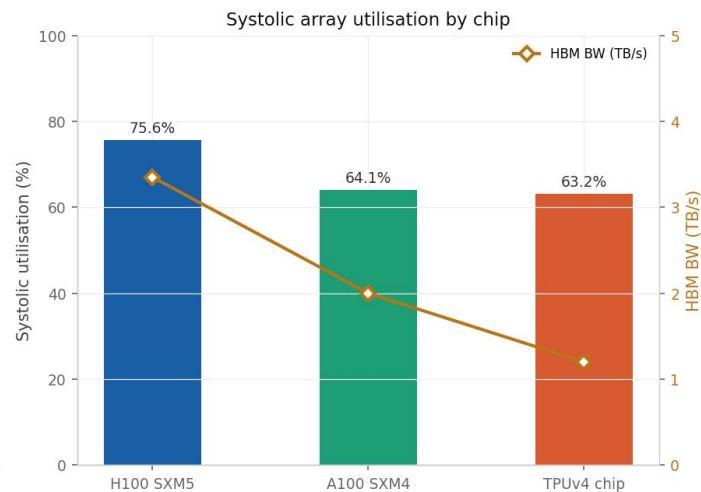
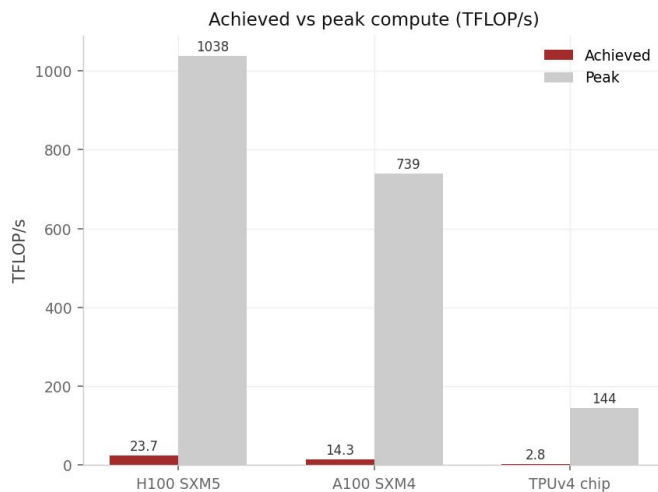
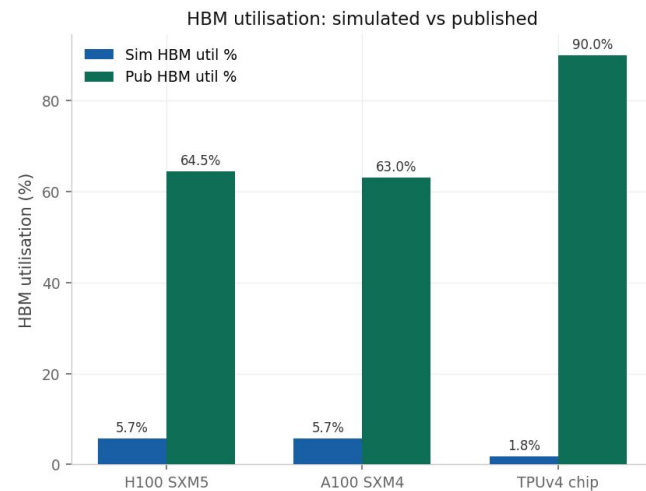
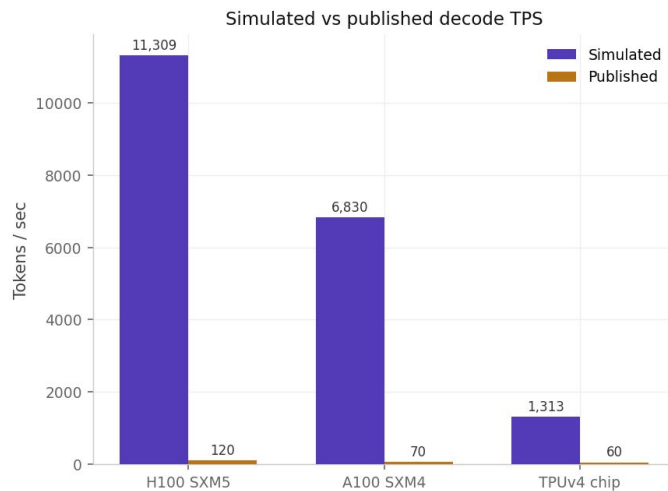


- Although not shown, using GQA drastically reduces memory usage. It is interesting (although expected) to see that throughput doesn't increase.
- Also notice, as we grouped more kv heads to be shared, this didn't make any difference. This seems to be because we were not memory bottlenecked and didn't reach a context size where this made a difference. Testing this would be something to try next.

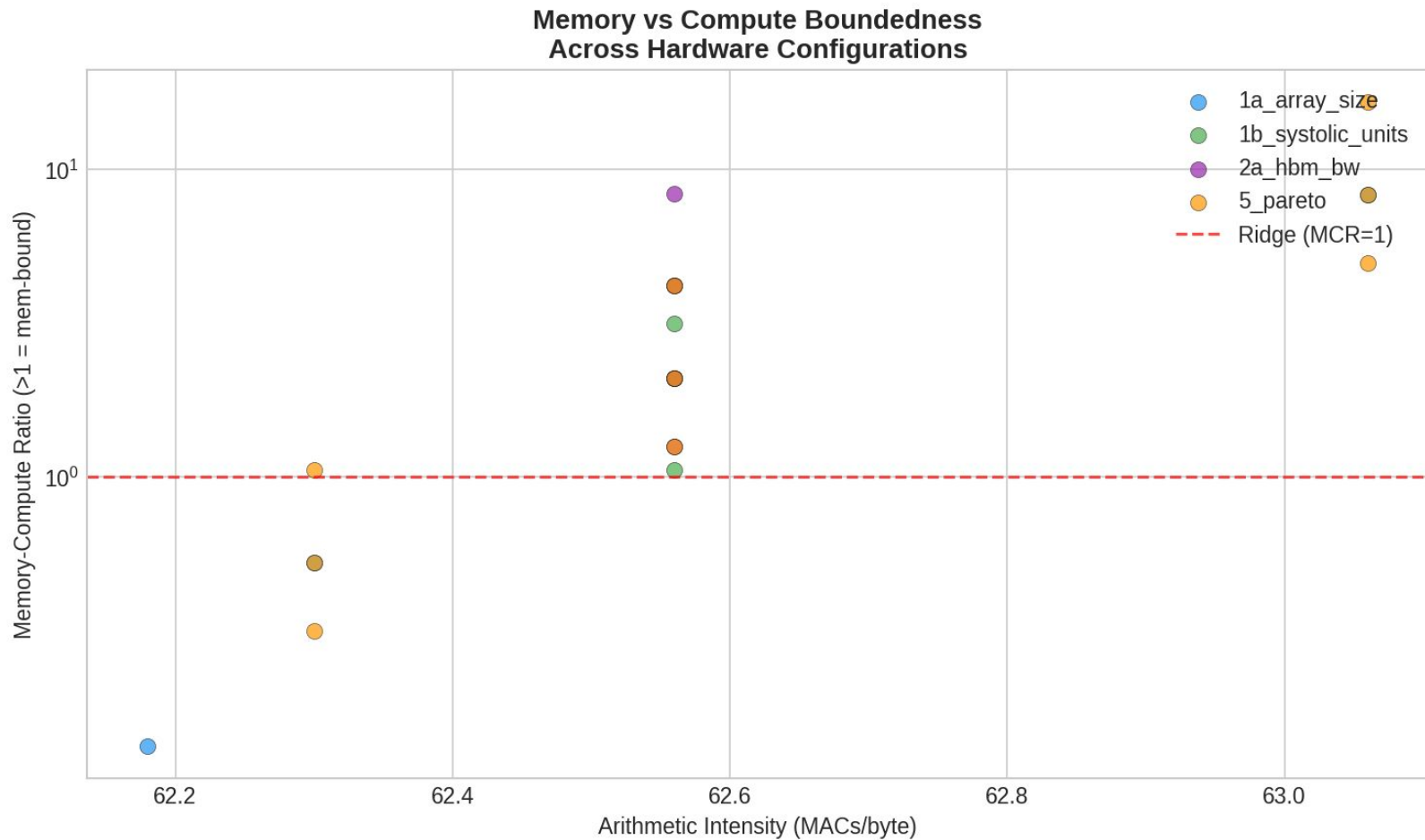
Group 5: Pareto Frontier Curve (1B Model)

5: Pareto — Array Size × HBM BW Trade-off Space





MCR



Case Study :Edge devices vs Data Center

DATACENTER

Systolic Array
256×256 × 8 units, 6 vector cores

Peak Compute
2,097 TFLOPS

Memory BW
3.35 TB/s HBM3

Ridge Point
626 ops / byte

SRAM Buffer
16 MB

Hardware:
The Scale
Gap

170×
Compute

33.5×
Mem BW

5.1×
Ridge Pt.

EDGE DEVICE

Systolic Array
64×64 × 1 unit, 2 vector cores

Peak Compute
12.3 TFLOPS

Memory BW
0.10 TB/s LPDDR5X

Ridge Point
123 ops / byte

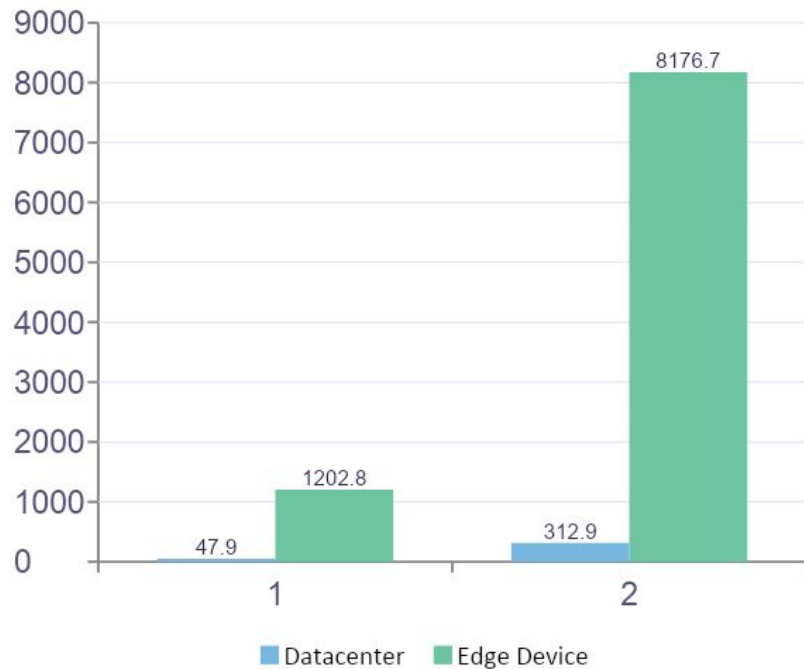
SRAM Buffer
1 MB

$$\text{Ridge Point} = \frac{\text{Peak TFLOPS}}{\text{Peak Memory BW (TB/s)}}$$

Case Study :Edge devices vs Data Center - Inference

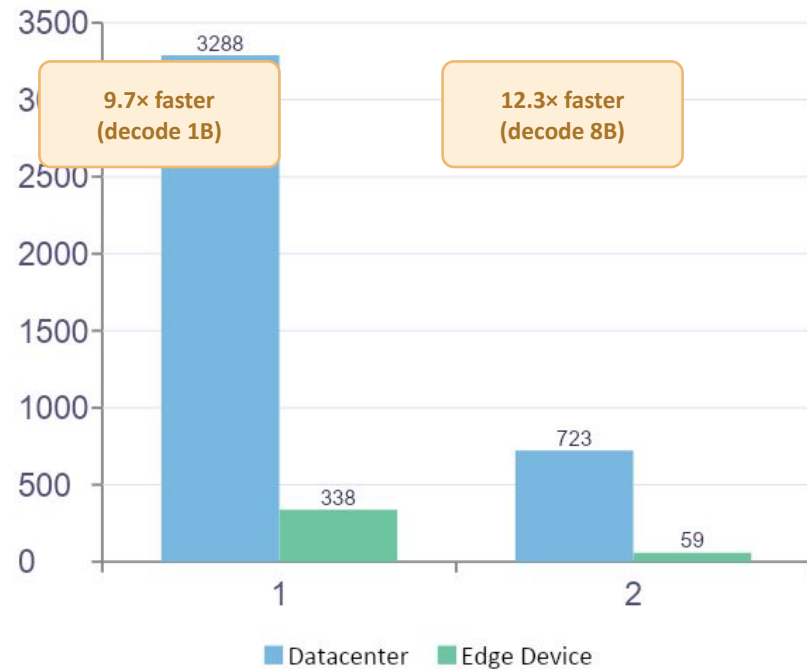
Prefill: TTFT per Input Token

lower is better · $\mu\text{s} / \text{token}$



Decode Throughput

higher is better · $\text{tokens} / \text{second}$



* 8B model: Datacenter ran 2,048-token prompt; Edge ran 512-token prompt (longer caused OOM)

Simulator Metrics and KPI Indicators

<https://docs.google.com/document/d/1jC7XII4TPYu7f7tUXdkSz5swLOwloRjyQSmKc-cEO7c/edit?tab=t.0>

Detailed Group configs

<https://docs.google.com/document/d/1eJZ4XdhvbAnunq0noMkVX--P0bdccVKVtRBhMN-bm3s/edit?tab=t.0>

Try it live!

Takeaways

Insight 1

- Arithmetic intensity is set by the workload, not the hardware, 62 to 63 MACs/byte across every config
- You cannot buy your way out of memory-boundedness by adding compute

Insight 2

- Prefill and decode respond to the same knobs in opposite directions
- Bigger array helps prefill, hurts decode. GQA saves prefill 13%, moves decode 2.4%
- There is no single lever that optimises both

Insight 3

- The one thing that consistently helps both phases is bandwidth
- 128×128 high-BW beats 512×512 low-BW, compute area is less valuable than bandwidth
- It compounds: longer context means more KV cache, means more bandwidth pressure every token

Future Work

Calibrate against industry standards for 1B model

8B/70B numbers only were found